

Soft Computing

BY

DR. REKHA SAHU



Silicon University

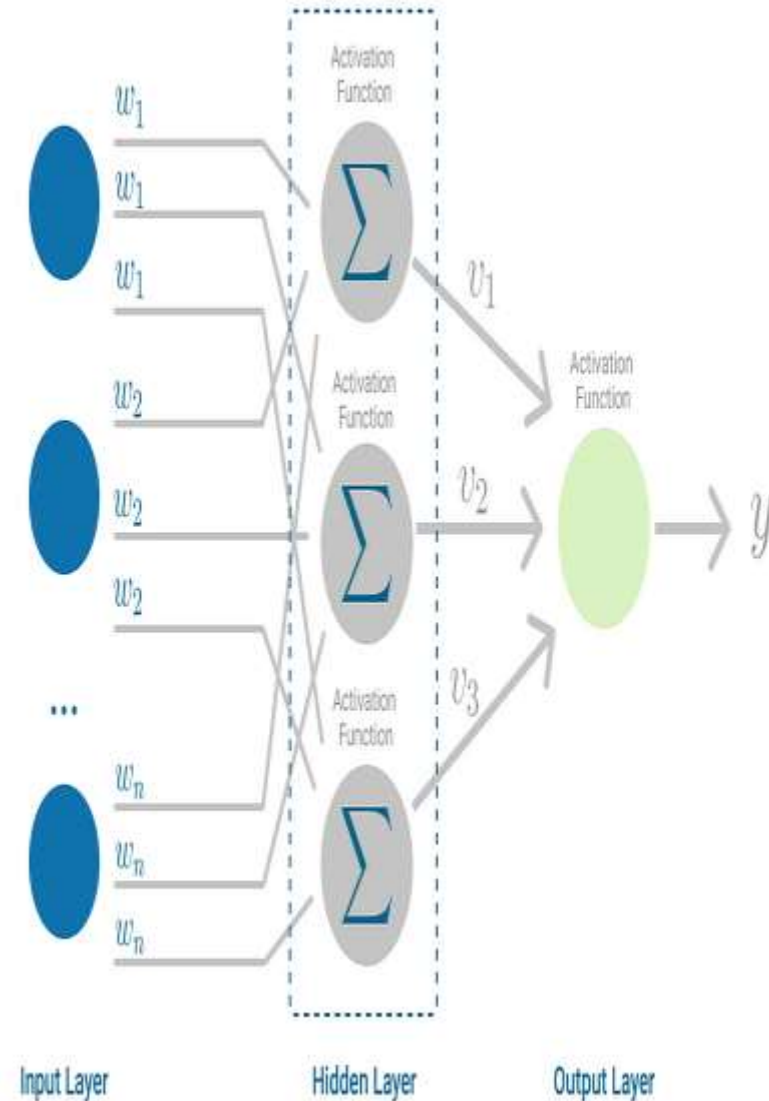
Module - 4

Detailed Syllabus

Module-#	Topics	Hours
Module-1	Fuzzy Logic: Basic definition and terminology of fuzzy set, Set theoretic operations, T-norm, T-conorm, Membership function formulation and parameterization, Extension principle, Fuzzy relations, Linguistic variables, Fuzzy if-then rules, Compositional rule of inference, Fuzzy reasoning, Fuzzy inference systems, Mamdani fuzzy models, Defuzzification, Sugeno and Tsukamoto fuzzy models.	10 Hours
Module-2	Genetic Algorithm: Introduction, Working cycle of a GA, Binary Coded GA, GA-parameter setting, Constraint handling GA, Advantages and disadvantages of GA, Some specialized GA (Real Coded GA).	8 Hours
Module-3	Neural Network - I: Introduction, Models of a neuron, Network architecture, Knowledge representation; Learning process - Error correction learning, Memory based learning, Hebbian learning, Competitive learning, Boltzmann learning, Learning with and without a teacher; Single layered learning - Least Mean Square algorithm, Perceptron, ADALINE, MADALINE.	10 Hours
Module-4	Neural Network - II: Multilayer perceptron - Back-propagation algorithm, XOR problem; Self-organizing Maps - Two basic feature mapping models, SOM algorithm, Radial Basis Function Network, Introduction to ART.	8 Hours
Module-5	Hybrid Systems: Combination of Genetic Algorithms with Fuzzy Logic or Neural Networks, Combination of Neural Network and Fuzzy Logic.	6 Hours
Total		42 Hours

Multilayer perceptron

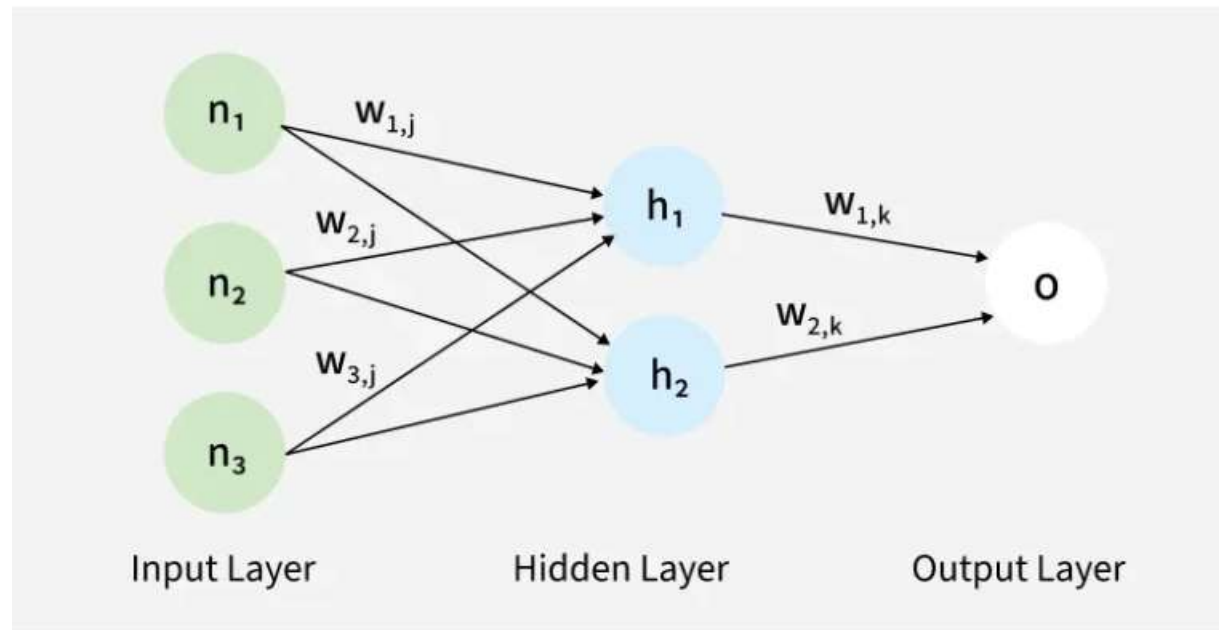
- Multilayer perception is a neural network
- The mapping between inputs and output is balance between linear and non-linear.
- It has input and output layers, and one or more hidden layers with many neurons stacked together.
- The neuron must have an activation function that imposes a threshold.
- Neurons can use any arbitrary activation function.



Feedforward Algorithm

- ❑ Inputs are combined with the initial weights in a weighted sum and subjected to the activation function.
- ❑ Each linear combination is propagated to the next layer.
- ❑ Each layer is feeding the next one with the result of their computation, their internal representation of the data. This goes through the hidden layers to the output layer.
- ❑ If the algorithm only computed the weighted sums in each neuron, propagated results to the output layer, and stopped there.

When it is needed to minimize the cost function, get optimum weights by learning then Backpropagation is needed.



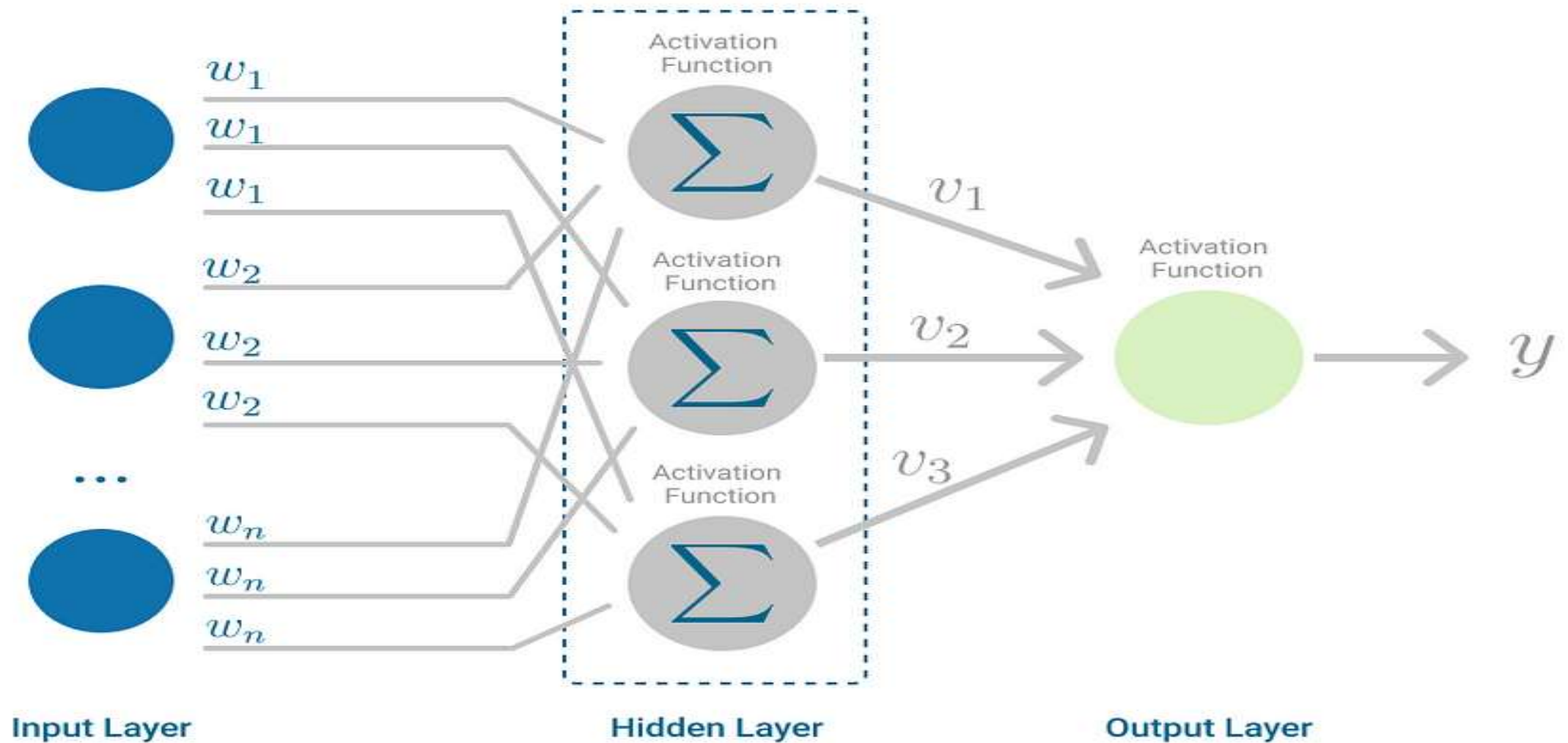
The forward pass using weights and biases

Backpropagation Algorithm

- Backpropagation is the learning mechanism that allows the Multilayer Perceptron to iteratively adjust the weights in the network, with the goal of minimizing the cost function.

- There is one hard requirement for backpropagation to work properly:

The function that combines inputs and weights in a neuron, and the threshold function, must be differentiable. These functions must have a bounded derivative (as gradient descent optimization is used).

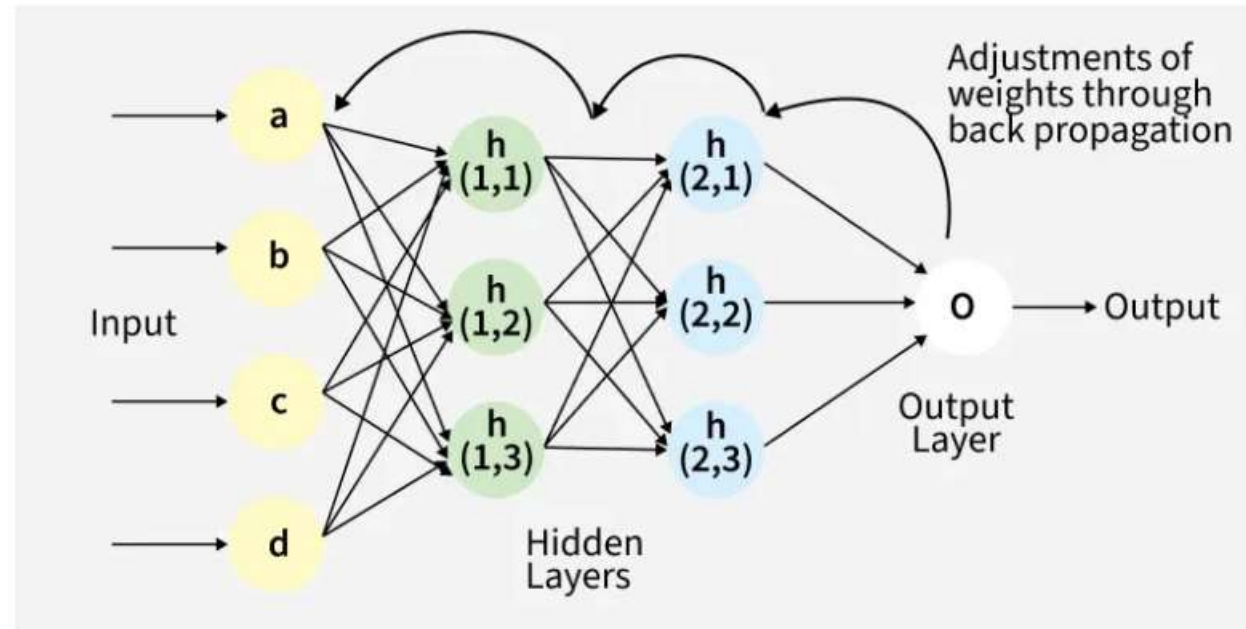


1. Feedforward | Mean Squared Error (MSE) computed

2. Backpropagation | Gradient is computed

- In each iteration, after the weighted sums are forwarded through all layers, the gradient of the Mean Squared Error is computed across all input and output pairs.
- Then, to propagate it back, the weights of the first hidden layer are updated with the value of the gradient $\Delta w(t)$ and back to the starting point of the neural network.
- This process keeps going until gradient for each input-output pair has converged, i.e., the newly computed gradient hasn't changed more than a specified convergence threshold, compared to the previous iteration.

This is called learning of weights.



Fig(a) A simple illustration of how the backpropagation works by adjustments of weights

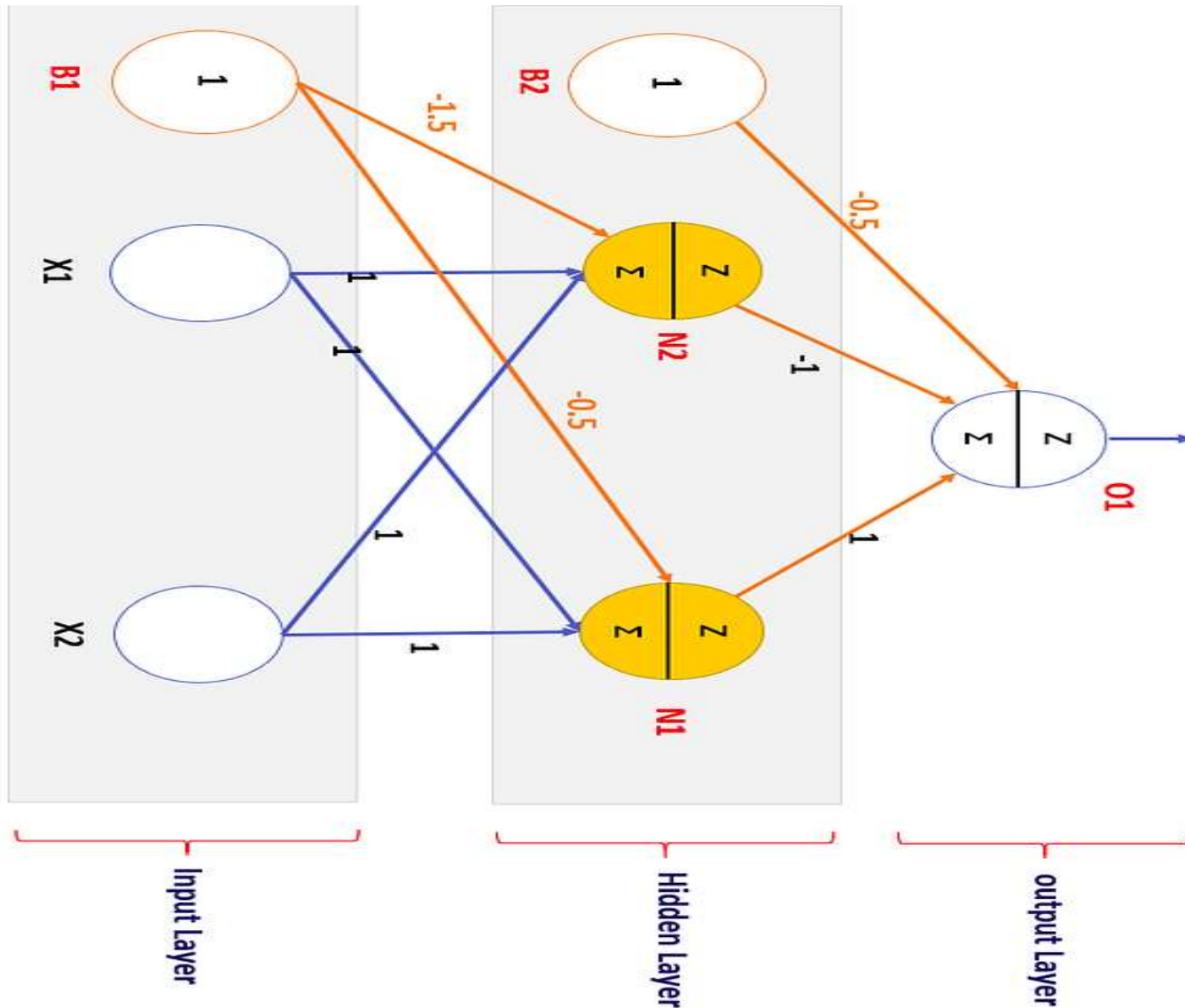
XOR gate with multilayer perception

The truth table for a two input XOR gate as follows:

X1	X2	Output
0	0	0
1	0	1
0	1	1
1	1	0

We have to model a neural network so that we can get output corresponding to input x1 and x2 as shown in above table.

The MLP architecture for the XOR gate (representation using perceptron) is



The circles are neurons(O_1 , N_1 , N_2 , X_1 , X_2), and the orange and blue lines with numbers are the representation of input direction with weights. The numbers on the arrows represent weights. B_1 and B_2 represent biases.

$$N1 = X_2 \times 1 + X_1 \times 1 + B_1 \times (-0.5)$$

$$N2 = X_2 \times 1 + X_1 \times 1 + B_1 \times (-1.5)$$

$$O1 = N_1 \times 1 + N_2 \times (-1) + B_2 \times (-0.5)$$

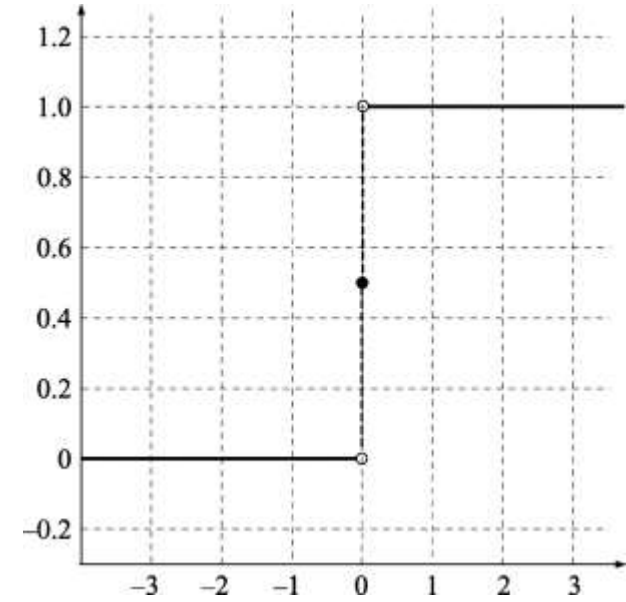
$$\text{The activation function } f(z) = \begin{cases} 1, & \text{if } z \geq 1 \\ 0, & \text{if } z < 1 \end{cases}$$

The step function (z) ,triggers only if the weighted sum is 1 or greater than 1.

if $X_1 * W_1 + X_2 * W_2 \geq 1$, *then*, *Step - Fuction*($X_1 * W_1 + X_2 * W_2$) = 1
if $X_1 * W_1 + X_2 * W_2 < 1$, *then*, *Step - Fuction*($X_1 * W_1 + X_2 * W_2$) = 0

OR

The activation function $f(z) = \begin{cases} 1, & \text{if } z \geq 1 \\ 0, & \text{if } z < 1 \end{cases}$



Calculation of XOR gate output:

Row 1 the XOR gate truth table: if $X_1 = 0$ and $X_2 = 0$, the output should be 0.

For hidden layer neuron N1:

$$\begin{aligned} N_1 &= X_2 * 1 + X_1 * 1 + B_1 * (-0.5) \\ &= 0 * 1 + 0 * 1 - 0.5 = -0.5 \end{aligned}$$

So, $\text{step_function}(-0.5) = 0$, output of **N1 = 0**

For the hidden layer neuron N2:

$$\begin{aligned} N_2 &= X_2 * 1 + X_1 * 1 + B_1 * (-1.5) \\ &= 0 * 1 + 0 * 1 - 1.5 = -1.5 \end{aligned}$$

So, $\text{step_function}(-1.5) = 0$, output of **N2 = 0**

For the output neuron O1 :

$$\begin{aligned} O_1 &= N_2 * (-1) + N_1 * 1 + B_2 * (-0.5) \\ &= (0) * (-1) + (0) * 1 + 1 * (-0.5) \\ &= 0 - 0 - 0.5 = -0.5 \end{aligned}$$

So, $\text{step_function}(-0.5) = 0$, output of **O1 = 0**

Thus, XOR truth table first row is matched.

Row 2 the XOR gate truth table: if $X_1 = 1$ and $X_2 = 0$, the output should be 1 .

For hidden layer neuron N1: $N_1 = X_2 * 1 + X_1 * 1 + B_1 * (-0.5) = 0 * 1 + 1 * 1 - 0.5 = 1 - 0.5 = 0.5$

So , $\text{step_function}(0.5) = 1$,output of **N1 = 1**

For the hidden layer neuron N2: $N_2 = X_2 * 1 + X_1 * 1 + B_1 * (-1.5) = 0 * 1 + 1 * 1 - 1.5 = 1 - 1.5 = -0.5$

So , $\text{step_function}(-1.5) = 0$,output of **N2 = 0**

For the output neuron O1 :
$$O_1 = N_2 * (-1) + N_1 * 1 + B_2 * (-0.5)$$
$$= (0) * (-1) + (1) * 1 + 1 * (-0.5) = 0 - 1 - 0.5 = -0.5$$

So, $\text{step_function}(-0.5) = 0$,output of **O1 = 0**

Thus, XOR truth table second row is matched.

Row 3 the XOR gate truth table: if $X_1 = 0$ and $X_2 = 1$, the output should be 1 .

For hidden layer neuron N1: $N_1 = X_2 * 1 + X_1 * 1 + B_1 * (-0.5) = 1 * 1 + 0 * 1 - 0.5 = 1 - 0.5 = 0.5$

So , $\text{step_function}(0.5) = 1$,output of **N1 = 1**

For the hidden layer neuron N2: $N_2 = X_2 * 1 + X_1 * 1 + B_1 * (-1.5) = 1 * 1 + 0 * 1 - 1.5 = 1 - 1.5 = -0.5$

So , $\text{step_function}(-1.5) = 0$,output of **N2 = 0**

For the output neuron O1 $O_1 = N_2 * (-1) + N_1 * 1 + B_2 * (-0.5)$
 $= (0) * (-1) + (1) * 1 + 1 * (-0.5) = 0 - 1 - 0.5 = -0.5$

So, $\text{step_function}(-0.5) = 0$,output of **O1 = 0**

Thus, XOR truth table third row is matched.

Row 4 the XOR gate truth table: if $X_1 = 1$ and $X_2 = 1$, the output should be 0 .

For hidden layer neuron N1: $N_1 = X_2 * 1 + X_1 * 1 + B_1 * (-0.5) = 1 * 1 + 1 * 1 - 0.5 = 1 + 1 - 0.5 = 1.5$

So ,step_function(1.5) = 1 ,output of **N1 = 1**

For the hidden layer neuron N2: $N_2 = X_2 * 1 + X_1 * 1 + B_1 * (-1.5) = 1 * 1 + 1 * 1 - 1.5 = 1 + 1 - 1.5 = 0.5$

So , step_function(0.5) = 1 ,output of **N2 = 1**

For the output neuron O1 :
$$O_1 = N_2 * (-1) + N_1 * 1 + B_2 * (-0.5)$$
$$= (1) * (-1) + (1) * 1 + 1 * (-0.5) = -1 + 1 - 0.5 = -0.5$$

So, step_function(- 0.5) = 0 ,output of **O1 = 0**

Thus, XOR truth table fourth row is matched.

NOTE:

- The weights used here are predetermined.
- In a real-world situation, we have to use a method called backpropagation to train the multilayer perceptron and after training, we will get the weights.
- The above multilayer perceptron(MLP), each row of the XOR gate truth table has given the correct output.

Self-Organizing Maps (SOMs)

SOMs are a type of unsupervised neural network that can be used for clustering and visualization of high-dimensional data. SOMs map high-dimensional data to lower-dimensional grids (often 2D) while preserving the topological structure, meaning that similar data points will be placed near each other in the output map.

Here are some common applications of Self-Organizing Maps:

1. **Data Clustering:**

SOMs can be used for clustering large datasets. They group similar data points together, which helps identify patterns and outliers in the data. This is useful in fields like:

- Market segmentation
- Customer behavior analysis
- Image segmentation

2. Dimensionality Reduction:

SOMs reduce the dimensionality of complex datasets, making it easier to visualize and understand. They help in visualizing high-dimensional data in a 2D or 3D grid.

3. Feature Extraction:

SOMs can learn features from the data and represent these features in a low-dimensional grid. This is often useful in tasks such as:

- Speech recognition
- Image recognition
- Text mining (where they can be applied to reduce the dimensionality of the term-document matrix)

4. **Visualization of High-Dimensional Data:**

One of the most notable applications of SOM is data visualization. It projects high-dimensional data onto a 2D grid while preserving its topological relationships. This allows users to:

- Visualize clusters in large datasets
- Identify relationships between different features
- Visualize the results of unsupervised learning processes

5. **Anomaly Detection:**

SOMs can be trained on normal data to detect outliers or anomalies. Once the model is trained, data points that do not fit the patterns learned by the map can be flagged as anomalies.

6. **Time Series Prediction:**

SOMs can also be applied to time-series data. They are helpful in recognizing patterns and temporal trends. This is useful in applications like:

- Stock market prediction
- Weather forecasting
- Health monitoring (for example, ECG signal analysis)

7. Image and Video Processing:

SOMs are used in image and video analysis to group similar regions or objects, and for dimensionality reduction in image compression. They can also be applied to:

- Image clustering
- Texture recognition
- Object tracking in video data

8. Bioinformatics:

In bioinformatics, SOMs are used to identify clusters of genes or proteins with similar expression patterns, and for visualization of complex biological data, such as DNA or protein sequences.

9. Robotics:

In robotics, SOMs are used for tasks like:

- Self-localization (mapping the robot's environment)
- Path planning
- Object recognition and classification

10. **Market Research and Consumer Segmentation:**

SOMs are applied to group consumers based on purchasing behavior, demographic data, or preferences. This helps companies in:

- Identifying market segments
- Targeted marketing
- Product recommendations

Two basic feature mapping models are as follows.

Self Organizing Map (SOM)

Radial Basis Function (RBF)

Self Organizing Map (or Kohonen Map or SOM)

- **Self Organizing Map** is a type of Artificial Neural Network.

It follows an unsupervised learning approach and trained its network through a competitive learning algorithm.

- SOM is used for clustering and mapping (or dimensionality reduction) techniques to map multidimensional data onto lower-dimensional which allows people to reduce complex problems for easy interpretation.

A **Self-Organizing Map (SOM)** has **two layers**:

1. Input Layer:

1. This layer receives the **raw data**.
2. Each neuron in the input layer **corresponds to one feature** (dimension) of the data.
3. If your data has 4 features, the input layer will have **4 neurons**.

2. Output Layer (or Kohonen Layer):

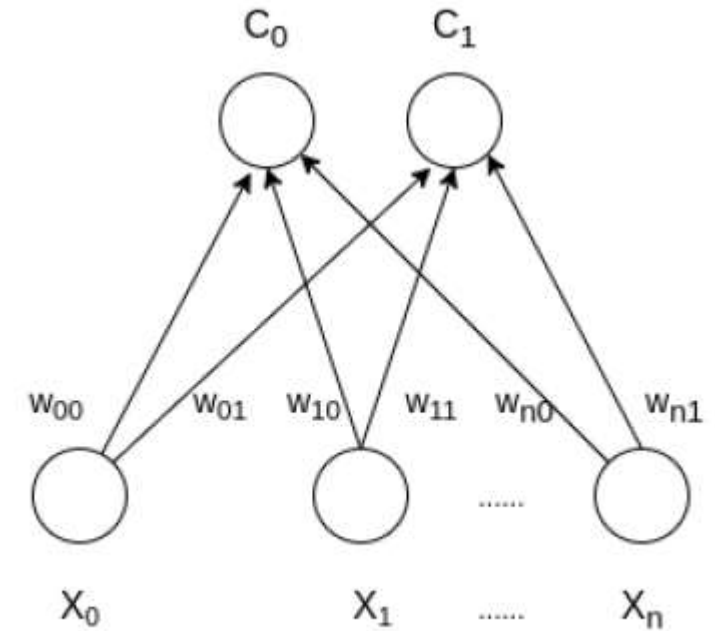
1. **A grid of neurons (usually 2D) that organizes itself during training.**
2. Each neuron has a **weight vector** with the same dimension as the input data.
3. The grid can be **rectangular** or hexagonal.

3. Feeding the input: Data is passed to the input layer.

4. Finding the BMU (Best Matching Unit): The neuron whose weight vector is closest to the input vector is chosen as the winner.

5. Updating weights: The BMU and its neighboring neurons update their weights to get closer to the input vector (this is where learning happens).

The architecture of the Self Organizing Map with two clusters and n input features.



Key concepts:

- Topology preservation: Nearby points in the input space stay close on the map.
- Dimensionality reduction: High-dimensional data turns into a 2D grid.
- Clustering: Similar patterns cluster in specific regions of the map.

Explanation of SOM

Let's say an input data of size (m, n) where m is the number of training examples and n is the number of features in each example. First, it initializes the weights of size (n, C) where C is the number of clusters. Then iterating over the input data, for each training example, it updates the winning vector (weight vector with the shortest distance e.g. Euclidean distance from training example).

α is a learning rate at time t ,

j denotes the winning vector,

i denotes the i^{th} feature of training example

k denotes the k^{th} training example from the input data.

After training the SOM network, trained weights are used for clustering new examples. A new example falls in the cluster of winning vectors.

Algorithm for Training:

Step 1: Initialize the weights w_{ij} random value may be assumed. Initialize the learning rate α .

Step 2: Calculate squared Euclidean distance.

$$D(j) = (\sum (w_{ij} - x_i)^2)^{0.5} \quad \text{where } i=1 \text{ to } n \text{ and } j=1 \text{ to } m$$

Step 3: Find index J , when $D(j)$ is minimum that will be considered as winning index.

Step 4: For each j within a specific neighborhood of j and for all i , calculate the new weight.

$$w_{ij}(\text{new}) = w_{ij}(\text{old}) + \alpha[x_i - w_{ij}(\text{old})]$$

Step 5: Update the learning rule by using :

$$\alpha(t+1) = 0.5 * t$$

Step 6: Test the Stopping Condition.

Radial Basis Function (RBF)

The typical architecture of a radial basis functions neural network consists of an input layer, hidden layer, and summation layer.

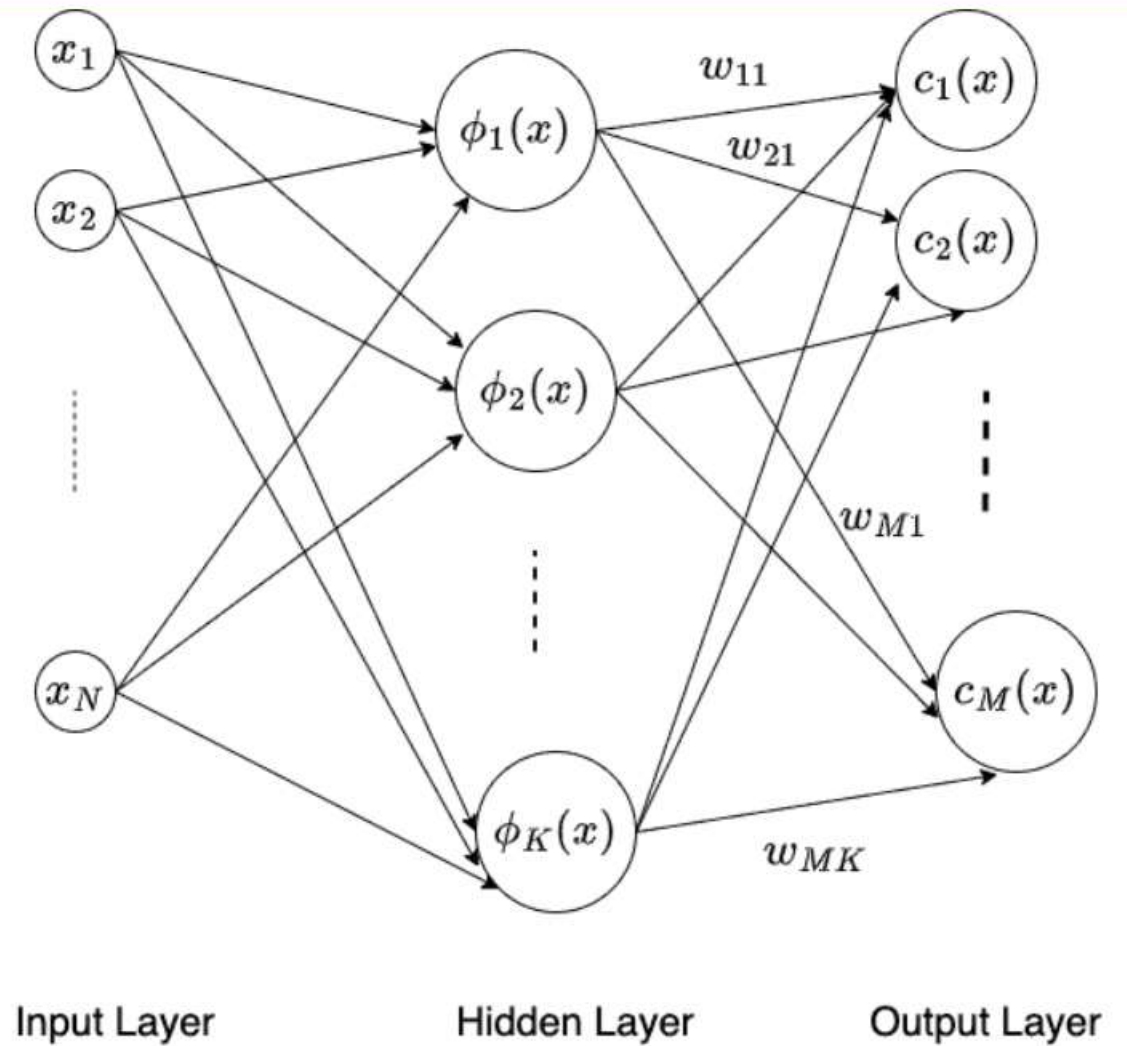
- Input Layer :The input layer consists of one neuron for every predictor variable. The input neurons pass the value to each neuron in the hidden layer.
- Hidden Layer: The hidden layer contains a variable number of neurons (the ideal number determined by the training process). Each neuron comprises a radial basis function centered on a point. The number of dimensions coincides with the number of predictor variables. The radius or spread of the RBF function may vary for each dimension.

$$\phi_i(x) = e^{-\frac{|x-\bar{x}|^2}{2\sigma^2}}$$

- Output layer

The network's output comprises a set of nodes for each category to classify. Each output node computes a score for the concerned category. Generally, we take a classification decision by assigning the input to the category with the highest score.

The score is calculated based on a weighted sum of the activation values from all RBF neurons. It usually gives a positive weight to the RBF neuron belonging to its category and a negative weight to others. Each output node has its own set of weights.



Steps

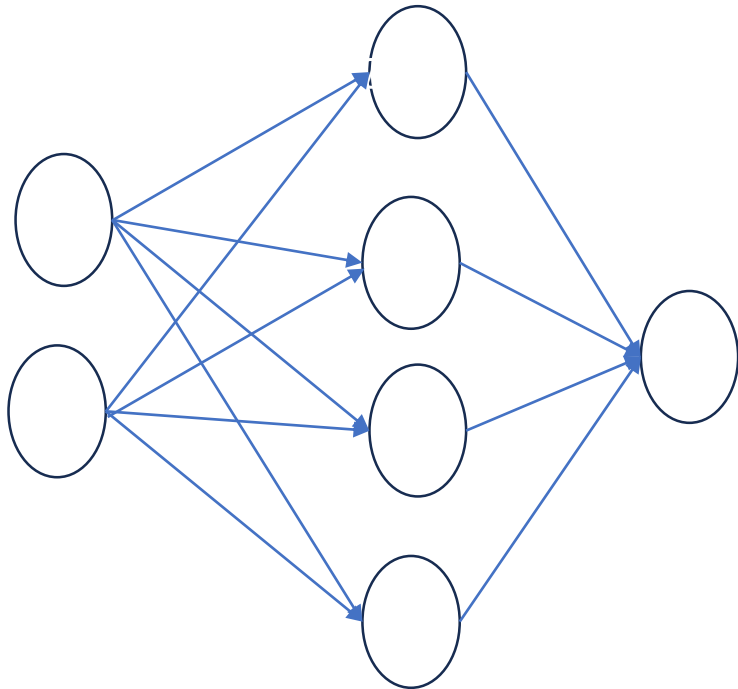
1. RBFNs start with an input vector. This vector is fed into the input layer of the network.
2. The network also has a hidden layer, which comprises radial basis function (RBF) neurons.
3. Each of these RBF neurons has a center, and they measure how close the input is to their center. They do this using a special function called a Gaussian transfer function. The output of this function is higher when the input is close to the neuron's center and lower when the input is far away.
4. The outputs from the hidden layer are then combined in the output layer. Each node in the output layer corresponds to a different category or class of data. The network determines the input's class by calculating a weighted sum of the outputs from the hidden layer.
5. The final output of the network is a combination of these weighted sums, which is used to classify the input.

Construct the RBFNN for XOR operation. Define four hidden layer neurons with Gaussian RBF

$$\phi_i(x) = e^{-\frac{(x - c_i)^2}{r^2}},$$

$$c_1 = (0, 0), c_2 = (0, 1), c_3 = (1, 0), c_4 = (1, 1), r^2 = 2.$$

Consider the XOR gate with following architecture. From hidden layer to output layer weight values are $w_1 = -0.8, w_2 = 0.9, w_3 = 0.9, w_4 = 0.8$.



$$\text{Activation function } f(x) = \begin{cases} 1, & \text{if } x > 0 \\ 0, & \text{if } x \leq 0 \end{cases}$$

What is backpropagation? Check whether backpropagation needed here or not.

2. Define different types of hybridization with examples.